

HARMAN

HControl Protocol

Quick Start Guide

Version 1.23.0.0

Contents

Quick Start Guide Overview.....	3
Protocol Overview	3
Introduction to the HControl Messaging Structure	3
Message Structure Overview	3
Introduction to HControl devices	4
Objects.....	5
Parameters.....	5
Commands	6
HControl messaging.....	7
Examples of Messaging.....	7
Get and Set Messages in HControl	8
Overview ' get ' Messages	8
Overview of ' set ' Messages.....	8
Error Messages	9
How formats work with the data types.....	11
Understanding Normalized Messaging Format	13
Examples of using Format with set commands	13
Commands	15
Using ' exec ' Command in HControl	16
Subscriptions in HControl.....	17
Overview.....	17
Subscription Commands Overview	17
Descriptor file	19
Descriptor format	19

Quick Start Guide Overview

Protocol Overview

This Quick Start Guide is designed to help you quickly understand and use the key features of the [HControl Protocol](#), enabling you to get up and running with basic operations such as setting parameters, receiving parameter updates and executing commands. This guide will provide a concise overview of these operations, allowing you to efficiently start working with the protocol.

Introduction to the HControl Messaging Structure

The HControl Protocol establishes a framework for clear and efficient communication between devices and controllers within a network. Central to this framework is a structured approach to messaging, utilizing JSON objects to encapsulate commands and responses. This guide provides an in-depth look at the HControl message structure, ensuring a standardized format for interaction across various devices and systems.

Message Structure Overview

Command, Parameters and Replies

```
<command> { <parameters> } <LF>  
@<reply>
```

- **Command:** Specifies the action to be performed, encapsulated within a JSON object.
- **Parameters:** Detailed within JSON objects, parameters include "path", "format", and "value".
- **Replies:** Each command is paired with a corresponding reply, distinguished by an "@" symbol, to differentiate replies from requests.

Rules and Semantics

- **Case Sensitivity:** HControl maintains case sensitivity for messages, parameters, and attribute names, typically in lowercase.
- **Encoding Compliance:** HControl messages adhere to UTF-8 encoding as mandated by Internet Standard 90 (RFC 8259), ensuring universal readability and compatibility.

- **Termination:** Messages conclude with a line feed (LF) or carriage return (CR), emphasizing the need for a flat JSON structure without embedded line breaks. Embedded carriage return or line feed characters are excluded to prevent premature termination.

Request and Response Formats

HControl messages are meticulously crafted to ensure seamless interaction:

- **Request Message Structure:** Each request initiates with a specific command followed by a JSON object containing parameters relevant to that command and completes with a LF (line feed) or CR (carriage return).

Example:

```
command {"path": "...", "format": "..."} <LF>
```

- **Response Message Structure:** Responses are identifiable by an "at" sign ("@") prefix and include the information requested or an acknowledgment of the command executed.

Example:

```
@command {"path": "...", "format": "...", "value": "..."} <LF>
```

It is important to use the correct syntax, while the message parameters adheres to the JSON standards, the command itself is contained within a line feed.

Introduction to HControl devices

HControl devices are composed of a hierarchical object structure where each object can have parameters and commands as well as child objects.

Objects, parameters and commands are referenced using their path. The path is formed from the names of the objects, parameters and commands.

Path Parameter Significance

HControl messages offer flexibility in parameter settings, supporting individual or grouped parameters within a JSON object or array. This adaptability is crucial for detailed device configuration and control. The "path" parameter is pivotal, specifying the target device parameter or attribute within the hierarchical device descriptor. The "path" parameter utilizes a forward slash ("/") to denote hierarchy, facilitating device object and parameter interpretations.

The following characters are not valid for any names which are part of a path:

\$ @ # ^ & " () [] \ / : ; ' < > { } | + = * % ! ` , ?

An example of a path is:

/configuration/device/name

The beginning / indicates this is the root of the path. Off the root there is an object named 'configuration' and within 'configuration' there is another object named 'device'. Within 'device' there is a parameter named 'name'.

Objects

Objects are containers for other objects, parameters and commands. Typically you do not perform operations directly on objects. Objects are useful in forming the structure of the device and organizing the parameters and commands in a logical form. The structure and names of the object form the 'path' when performing HControl commands.

Parameters

Parameters and their values are what forms the 'state' of a device. Through HControl you can read and write their values as well as be notified whenever the values change.

Parameters on the device are stored with different data types.

Type	Description
boolean	True or False value.
byte_array	A series of bytes, with a max defining the maximum length.
enum	A set of predefined values. Enumeration set must be defined.
string	Text values, UTF-8 encoded, with min and max denoting the minimum and maximum length of string allowed.
integer	A whole number, with min and max defining the range. (See appendix)
float	A decimal number, with min and max for range. (See appendix)
dstring	A delimited string, values separated by " " (eg, "Value1 Value2")

Parameter values can be read and written using different formats. The formats available are:

- **"norm" (Normalized):** Scales values between 0 and 1, ideal for parameters with a broad value range.
- **"number":** Represents values numerically, facilitating precision and unit specification.
- **"string":** Treats values as text, suitable for names, statuses, or other textual data.
- **"variant":** The most accurate representation of the parameter. This format type is useful in specific use cases and unlikely to be used in normal operation.

Invalid format specifications trigger an error response, underscoring the necessity for a valid format parameter value. Omitting the 'format' argument in HControl messages results in the 'variant' format being used.

Commands

HControl messaging

Importance of Lock-step Communication

Lockstep communication is an essential piece of the HControl Protocol, ensuring that **for each request sent, a corresponding reply is received before the next request is issued**. This mechanism is vital for:

- **Preventing Overload:** Protects devices, especially those with limited processing capabilities, from being overwhelmed by simultaneous requests.
- **Maintaining Order:** Ensures commands are processed in the sequence they are received, crucial for operations requiring sequential execution.
- **Adaptive Command Processing:** Allows devices to process commands at their own pace, supported by the lockstep mechanism to manage command queues effectively.

Examples of Messaging

Simple Request and Reply

This example demonstrates the basic message structure using the `get` command to retrieve the manufacturer name of a device. The `path` parameter specifies the target device attribute within the hierarchical device descriptor. Detailed information about the `get` command can be found later in the documentation. This illustration aims to show how the messaging structure is formed.

Request:

```
get {"format":"string","path":"/configuration/device/manufacturer"} <LF>
```

Response:

```
@get {"format":"string","path":"/configuration/device/manufacturer","value":"AMX"} <LF>
```

Get and Set Messages in HControl

Overview 'get' Messages

The **"get"** command in the HControl protocol is used to retrieve the current value of a parameter from a device. This command specifies the path to the desired parameter and optionally the format in which the reply should be provided. The device responds with the current value of the parameter in the requested format or returns an error if the request cannot be fulfilled. The "format" parameter allows you to specify how the value should be returned, and it will be discussed in the ["Format Parameter"](#) section after the **get** and **set** commands.

Below is a table that illustrates various scenarios for using the **"get"** command, along with examples of requests and their corresponding replies.

Scenario	Example Request	Example Reply	Description
Parameter with Format	<pre>get { "path": "/channel1/mute", "format": "integer" }</pre>	<pre>@get { "path": "/channel1/mute", "format": "integer", "value": 0 }</pre>	Retrieve the integer value for mute on channel 1 as a number.
Object Parameters	<pre>get { "path": "/channel1" }</pre>	<pre>@get [{ "path": "/channel1/gain", "format": "norm", "value": 0.5 }, { "path": "/channel1/mute", "format": "boolean", "value": 0 }]</pre>	Retrieve all parameters under the "channel 1" object.

Overview of 'set' Messages

The **"set"** command in the HControl protocol is a fundamental operation used to modify the value of one or more parameters on a device. Unlike the **"get"** command, which retrieves parameter values, **"set"** commands update the parameter values based on the provided path and value. This command is essential for controlling device behavior, adjusting settings, and applying configurations dynamically.

Command Structure

The **"set"** command in the HControl protocol is versatile and can be issued in various forms to update single parameters. It is important to understand that the value returned by the device is the actual value that has been set on the device, not necessarily the value that was issued in the command. This discrepancy occurs because the device may enforce rules, such as maximum allowable values, which can override the commanded value. For example, if a parameter's maximum value is set to 5 and a

command attempts to set it to 10, the device will set the parameter to 5 and return this value, reflecting the enforced rule.

Examples and Usage

Below is a table illustrating the use of the "set" command.

Scenario	Example Request	Example Reply	Description
Single Parameter with Format	<code>set { "path": "/channel1/gain", "format": "norm", "value": 0.5 }</code>	<code>@set { "path": "/channel1/gain", "format": "norm", "value": 0.5 }</code>	Sets the gain for channel 1 with a normalization format.
Error Handling	<code>set { "path": "/nonexistent/param", "value": 1 }</code>	<code>@set { "path": "/nonexistent/param", "error": "Parameter not found" }</code>	Attempt to set a non-existent parameter results in an error.

Error Messages

Error handling is a fundamental component of the HControl protocol, ensuring clear and effective communication between devices. It addresses issues that arise during the execution of "set" commands, such as setting undefined parameters or data type mismatches. This section outlines typical error scenarios, with examples of commands that may trigger these errors and the corresponding replies that describe the error encountered.

Common Error Scenarios

Scenario	Description	Example Command / Expected Reply
Parameter Not Found	Occurs when attempting to set a parameter that is not defined in the device's descriptor.	<code>set { "path": "/invalid/path", "value": 0.5 }</code>
		<code>@set { "path": "/invalid/path", "error": "Parameter not found" }</code>
Invalid Value Type	Triggered by providing a value that mismatches the expected data type for the parameter.	<code>set { "path": "/config/setting", "value": "text" }</code>
		<code>@set { "path": "/config/setting", "error": "Invalid value type" }</code>

Scenario	Description	Example Command / Expected Reply
Value Out of Range	Occurs when a value is set beyond the parameter's defined minimum or maximum limits.	set { "path": "/config/volume", "value": 150 }
		@set { "path": "/config/volume", "error": "Value out of range" }
Invalid Format Specified	Using an unsupported format specifier in the command.	set { "path": "/config/mode", "format": "binary", "value": "010101" }
		@set { "path": "/config/mode", "error": "Invalid format specified" }
Missing Required Parameter	Omitting a required value or attribute in the set command.	set { "path": "/config/network" }
		@set { "path": "/config/network", "error": "Missing required parameter" }

How formats work with the data types

Parameter Type	Format	Example Request and Reply	Description
boolean	norm	set {"path":"/test/boolean","format":"norm","value":1} @set {"format":"norm","path":"/test/boolean","value":1}	Sets boolean to true (1) in normalized format.
	number	set {"path":"/test/boolean","format":"number","value":1} @set {"format":"number","path":"/test/boolean","value":1}	Sets boolean to true (1) in number format.
	string	set {"path":"/test/boolean","format":"string","value":"true"} @set {"format":"string","path":"/test/boolean","value":"true"}	Sets boolean to true in string format.
byte_array	string	set {"path":"/test/block","format":"string","value":"SGVsbG8sIFdvcmxkIQ=="} @set {"format":"string","path":"/test/block","value":"SGVsbG8sIFdvcmxkIQ=="}	Sets byte array with base64 string.
dstring	string	set {"path":"/test/dstring","format":"string","value":"A B C D E"} @set {"format":"string","path":"/test/dstring","value":"A B C D E"}	Sets DString using delimited values of "A B C D E"
float	norm	set {"path":"/test/float","format":"norm","value":0.5} @set {"format":"norm","path":"/test/float","value":0.5}	Sets float in normalized format.
	number	set {"path":"/test/float","format":"number","value":1.2} @set {"format":"number","path":"/test/float","value":1.2}	Sets float in number format.
	string	set {"path":"/test/float","format":"string","value":"1.2"} @set {"format":"string","path":"/test/float","value":"1.2"}	Sets float in string format.
enum	norm	set {"path":"/test/enum-2","format":"norm","value":0.5} @set {"format":"norm","path":"/test/enum-2","value":0.5}	Sets enum to corresponding value in normalized format.
	number	set {"path":"/test/enum-2","format":"number","value":2} @set {"format":"number","path":"/test/enum-2","value":2}	Sets enum to corresponding index in number format.
	string	set {"path":"/test/enum-2","format":"string","value":"delta"} @set {"format":"string","path":"/test/enum-2","value":"delta"}	Sets enum to corresponding string value.
integer	norm	set {"path":"/test/integer","format":"norm","value":0.5} @set {"format":"norm","path":"/test/integer","value":0.5}	Sets integer in normalized format.

Parameter Type	Format	Example Request and Reply	Description
	number	<pre>set {"path":"/test/integer","format":"number","value":25} @set {"format":"number","path":"/test/integer","value":25}</pre>	Sets integer in number format.
	string	<pre>set {"path":"/test/integer","format":"string","value":"25"} @set {"format":"string","path":"/test/integer","value":"25"}</pre>	Sets integer in string format.
string	number	<pre>set {"path":"/test/string","format":"number","value":0.5} @set {"format":"number","path":"/test/string","value":"0.5"}</pre>	Sets string to number format.

Understanding Normalized Messaging Format

The normalized (norm) messaging format are linearly mapped values, standardizing the representation of parameter values. This format expresses values as floating-point numbers between 0 and 1, inclusive, facilitating a uniform approach to parameter manipulation.

Examples of Normalized Values

Min	Max	Value	Format String	Result
0	3	1	norm	0.333333
-15dB	+15dB	5dB	norm	0.666666

Examples of using Format with set commands

Below are examples of how to use various formats to set parameter values within the HControl protocol. It includes explanations on using the descriptors to determine the parameter types and illustrates how to structure requests and interpret replies.

Parameter Type	Format	Example Request and Reply	Description
boolean	norm	set {"path":"/test/boolean","format":"norm","value":1} @set {"format":"norm","path":"/test/boolean","value":1}	Sets boolean to true (1) in normalized format.
	number	set {"path":"/test/boolean","format":"number","value":1} @set {"format":"number","path":"/test/boolean","value":1}	Sets boolean to true (1) in number format.
	string	set {"path":"/test/boolean","format":"string","value":"true"} @set {"format":"string","path":"/test/boolean","value":"true"}	Sets boolean to true in string format.
byte_array	string	set {"path":"/test/block","format":"string","value":"SGVsbG8sIFdvcmxkIQ=="} @set {"format":"string","path":"/test/block","value":"SGVsbG8sIFdvcmxkIQ=="}	Sets byte array with base64 string.
dstring	string	set {"path":"/test/dstring","format":"string","value":"A B C D E"} @set {"format":"string","path":"/test/dstring","value":"A B C D E"}	Sets DString using delimited values of "A B C D E"
float	norm	set {"path":"/test/float","format":"norm","value":0.5} @set {"format":"norm","path":"/test/float","value":0.5}	Sets float in normalized format.
	number	set {"path":"/test/float","format":"number","value":1.2} @set {"format":"number","path":"/test/float","value":1.2}	Sets float in number format.
	string	set {"path":"/test/float","format":"string","value":"1.2"} @set {"format":"string","path":"/test/float","value":"1.2"}	Sets float in string format.
enum	norm	set {"path":"/test/enum-2","format":"norm","value":0.5} @set {"format":"norm","path":"/test/enum-2","value":0.5}	Sets enum to corresponding value in normalized format.
	number	set {"path":"/test/enum-2","format":"number","value":2} @set {"format":"number","path":"/test/enum-2","value":2}	Sets enum to corresponding index in number format.
	string	set {"path":"/test/enum-2","format":"string","value":"delta"} @set {"format":"string","path":"/test/enum-2","value":"delta"}	Sets enum to corresponding string value.
integer	norm	set {"path":"/test/integer","format":"norm","value":0.5} @set {"format":"norm","path":"/test/integer","value":0.5}	Sets integer in normalized format.

Parameter Type	Format	Example Request and Reply	Description
	number	<pre>set {"path":"/test/integer","format":"number","value":25} @set {"format":"number","path":"/test/integer","value":25}</pre>	Sets integer in number format.
	string	<pre>set {"path":"/test/integer","format":"string","value":"25"} @set {"format":"string","path":"/test/integer","value":"25"}</pre>	Sets integer in string format.
string	number	<pre>set {"path":"/test/string","format":"number","value":0.5} @set {"format":"number","path":"/test/string","value":"0.5"}</pre>	Sets string to number format.
	string	<pre>set {"path":"/test/string","format":"string","value":"Hello, World!"} @set {"format":"string","path":"/test/string","value":"Hello, World!"}</pre>	Sets string to given value.

Commands

In addition to parameter management, executing specific commands on devices is an essential aspect of the HControl Protocol. The "[Using 'exec' Command in HControl](#)" section explains how to perform direct actions through the protocol. Understanding this command broadens your control capabilities, allowing for more dynamic and responsive device interactions.

Using 'exec' Command in HControl

Overview

The "exec" command in the HControl protocol is a versatile tool designed for executing specific actions or commands on a device, offering a broad spectrum of direct interactions beyond standard parameter adjustments. This command is pivotal for tasks such as device resets, firmware updates, or executing custom device-supported commands, thereby broadening the scope of control for developers and system integrators.

Other examples using the 'exec' command for custom device-supported commands:

Description	Usage Example
Send Base64 encoded data from serial port 1 of a device	<code>exec {"path":"/serial/1", "command":"send", "arguments":{"data":"SGVsbG8="}}</code>
	<code>@exec</code>
Set flow control mode for the serial port 2 of a device	<code>exec {"path":"/serial/2", "command":"setFlowControl", "arguments":{"mode":"NONE"}}</code>
	<code>@exec</code>
Queue a timed pulse of IR by index on IR port 3 of a device	<code>exec {"path":"/ir/3", "command":"bufferedSendIr", "arguments":{"code":1}}</code>
	<code>@exec</code>
Queue a timed pulse invoked by name on IR port 3 of a device	<code>exec {"path":"/ir/3", "command":"bufferedSendNamedIr", "arguments":{"code":"PLAY"}}</code>
	<code>@exec</code>

Subscriptions in HControl

Overview

Subscriptions in the HControl protocol are designed to optimize device communication by allowing clients to monitor device state changes or events efficiently. This approach is particularly advantageous for real-time applications and automation systems where timely updates are essential.

Benefits of Subscription Commands

- **Efficient Resource Utilization:** Subscriptions reduce network traffic and CPU load by eliminating the need for constant polling, as updates are sent only when changes occur.
- **Real-time Feedback:** Clients receive instant notifications about state changes or events, enabling real-time monitoring and actions.
- **Customizable Monitoring:** Clients can tailor subscriptions to specific parameters or events, enhancing the monitoring process.
- **Scalability:** The subscription model supports a growing number of devices or parameters efficiently, outperforming traditional polling methods.
- **Error and Event Handling:** Subscriptions allow for immediate identification and resolution of issues, and enable automated actions in response to device feedback.

Subscription Commands Overview

Command	Description	Example Request
subscribe	Initiates subscription to changes for a specific parameter.	<pre>subscribe {"path":"/Device/Channel4PEQ/Band_1_Frequency", "format":"string"}</pre>
		<pre>@subscribe {"path":"/Device/Channel4PEQ", "format":"string", "value":{"Band_1_Frequency":"100Hz"}}</pre>
unsubscribe	Ends an active subscription to a single parameter.	<pre>unsubscribe {"path":"/Device/Channel4PEQ/Band_1_Frequency", "format":"string"}</pre>
		<pre>@unsubscribe {"path":"/Device/Channel4PEQ/Band_1_Frequency", "format":"string"}</pre>

Command	Description	Example Request
unsubscribe	Ends ALL active subscriptions	unsubscribe
		@unsubscribe

Subscription Message Flow

When a client subscribes to a parameter, the server responds with an **@subscribe** message containing the current values. Subsequent changes to the parameter are sent to the client using **publish** messages.

It is important to note that **publish** messages must be acknowledged with an **@publish** response from the client before another **publish** message can be sent. Failure to do so will halt further **publish** messages.

When you would like to end the active subscription, you can **unsubscribe** from either the path, the parent object or all subscriptions.

```
subscribe {"path":"/Device/Channel1/Gain","format":"norm"}
@subscribe {"format":"norm","path":"/Device/Channel1/Gain","value":0.2}

publish {"format":"norm","path":"/Device/Channel1/Gain","value":0.5}
@publish

unsubscribe {"path":"/Device/Channel1/Gain"}
@unsubscribe {"path":"/Device/Channel1/Gain"}
```

Descriptor file

Every HControl device has a descriptor file which describes the model of the device. It is a JSON object which contains all the objects, parameters and commands of a device. You can use the descriptor file to determine the location and therefore the path of all objects and parameters within the device.

A descriptor file maybe static or dynamic. Most devices are of a fixed architecture and their descriptor file is static. Some devices however are open-architecture such as the Soundweb OMNI processors, and so their descriptor file is dynamic.

For static descriptor files they should be readily available. For dynamic descriptor files, since they're not readily available you have to use AVX Architect to determine the paths of objects and parameters. AVX Architect contains a feature of copying HControl strings to the clipboard. You can access this feature from the right-click context menu on controls within a control panel.

Descriptor format

Consider the following JSON structure as an advanced example of a device descriptor. This example illustrates not only the basic structure but also how nested objects are utilized to represent complex device configurations.

```
{
  ".kind": "obj",
  ".class": "harman.amx.AcendoCoreDevice",
  "version": {
    ".kind": "param",
    ".class": "harman.common.version",
    ".type": "string",
    ".access": "c"
  },
  "configuration": {
    ".kind": "obj",
    ".class": "harman.common.configuration",
    ".scope": "config",
    "device": {
      ".kind": "obj",
      ".class": "harman.common.device",
      "classname": {
        ".kind": "param",
        ".type": "string",
        ".access": "c"
      },
    },
    "container": {
      ".kind": "param",
      ".type": "string"
    },
  },
  "descriptorlocation": {
    ".kind": "param",
    ".type": "string",
  },
}
```

```

        ".access": "c",
        ".max": 256
    },
    "devicestate": {
        ".class": "harman.common.devicestate",
        ".kind": "param",
        ".type": "string",
        ".access": "ro"
    },
    "exclusions": {
        ".kind": "param",
        ".type": "dstring",
        ".access": "c"
    },
    "manufacturer": {
        ".kind": "param",
        ".type": "string",
        ".access": "c"
    },
    "model": {
        ".kind": "param",
        ".type": "string",
        ".access": "c"
    },
    "name": {
        ".kind": "param",
        ".type": "string",
        ".min": 1,
        ".max": 64
    }
},
"network": {
    ".class": "harman.common.network",
    ".kind": "obj",
    "hostname": {
        ".kind": "param",
        ".type": "string",
        ".min": 1,
        ".max": 256,
        ".access": "ro",
        ".flags": [
            "uni"
        ]
    },
    "interface": {
        ".kind": "array",
        ".size": 3,
        ".items": [
            {
                ".kind": "obj"
            },
            {
                ".kind": "obj"
            },
            {
                ".kind": "obj"
            }
        ]
    }
}

```

```

    },
    "routes": {
      ".kind": "array",
      ".size": 1,
      ".prototype": {
        ".kind": "obj",
        "ip": {
          ".kind": "param",
          ".type": "string",
          ".class": "harman.common.ipv4"
        },
        "port": {
          ".kind": "param",
          ".type": "integer",
          ".class": "harman.common.port"
        }
      }
    }
  }
}

```

Key Components of a Device Descriptor

Component	Description
.kind	Indicates the JSON object type: "obj" for container objects, "param" for parameters, "array", or "command". It determines the role and behavior of the object within the device descriptor.
.class	This attribute assigns the object to a predefined class, which dictates its structure and the expected parameters it can contain.
.type	Defines the kind of data that the parameter can hold, "boolean", "byte_array", "enum", "string", "integer", "float" and "dstring", ensuring that values are interpreted correctly.
.access	Defines parameter accessibility: Read Only ("ro"), Write Only ("wo"), Read Write ("rw"), or constant ("c"). If .access is not defined, then it is assumed to be read write ("rw").
.scope	Specifies the context or category of the object, such as "config" for configuration settings, helping to organize the descriptor logically.
.min	The minimum value for the parameter if numeric. For string parameters this would be the minimum length of string allowed.
.max	The maximum value for the parameter if numeric. For string parameters this would be the maximum length of string allowed.
.default	For parameters which are constant, the default is the value of this constant parameter